

```
'setup': ["A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A",  
"A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A",  
"A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A",
```

Using **Pandas** is the industry standard for this because it allows you to easily analyze, filter, and clean your acoustic/electrical testing data. I've also clean-coded the raw dictionary format if you just want it as a native Python object.

This approach turns your raw data into a structured table, making it incredibly easy to run statistics or export to an Excel/CSV file.



```
import pandas as pd

# 1. Define the raw data dictionary
data = {
    'setup': ["A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A"],
    'env': ["Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind.", "Ind."],
    'spl': ["60 dB", "60 dB", "60 dB", "70 dB", "70 dB", "70 dB", "80 dB", "80 dB", "80 dB", "80 dB", "80 dB", "80 dB", "80 dB"],
    'noOfTrials': ["Round 1", "Round 2", "Round 3", "Round 1", "Round 2", "Round 3", "Round 1", "Round 2", "Round 3", "Round 1", "Round 2", "Round 3", "Round 1"],
    'dcVoltage': [0, 0, 0, 0, 0, 0, 0.003, 0.001, 0.001, 0.029, 0.047, 0.05, 1],
    'dcCurrent': [0, 0, 0, 0, 0, 0, 0.00003, 0.00001, 0.00001, 0.00029, 0.00047, 0.0005, 0.001],
    'dcPower': [0, 0, 0, 0, 0, 0, 9e-8, 1e-8, 1e-8, 0.00000841, 0.00002209, 0.000025, 0.0001]
}

# 2. Convert to DataFrame
df = pd.DataFrame(data)

# Optional: Clean up data types (e.g., stripping ' dB' to make SPL purely numerical)
df['spl_db'] = df['spl'].str.replace(' dB', '').astype(int)
```

```
# Optional: Clean up Trial column to be an integer
df['trial_no'] = df['noOfTrials'].str.replace('Round ', '').astype(int)

print(df.head(10))
```

Option 2: Native Python Dictionary (No External Libraries)

If you don't want to use Pandas and prefer standard Python, the data is already formatted correctly as a dictionary of lists. Below is the clean syntax wrapped nicely in a variable name.

Python



```
experimental_data = {
    'setup': ["A"] * 45 + ["B"] * 45,
    'env': (["Ind."] * 15 + ["Traf."] * 15 + ["Comm."] * 15) * 2,
    'spl': (["60 dB"] * 3 + ["70 dB"] * 3 + ["80 dB"] * 3 + ["90 dB"] * 3 + ["100 dB"] * 3) * 2,
    'noOfTrials': ["Round 1", "Round 2", "Round 3"] * 30,
    'dcVoltage': [
        0, 0, 0, 0, 0, 0, 0.003, 0.001, 0.001, 0.029, 0.047, 0.05, 1.132, 1.09,
        0, 0, 0, 0, 0, 0, 0.001, 0.001, 0.001, 0.002, 0.002, 0.002, 0.082, 0.0,
        0, 0, 0, 0, 0, 0, 0.001, 0.001, 0.002, 0.002, 0.002, 0.003, 0.203, 0.1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
    ],
    'dcCurrent': [
        0, 0, 0, 0, 0, 0, 0.00003, 0.00001, 0.00001, 0.00029, 0.00047, 0.0005,
        0, 0, 0, 0, 0, 0, 0.00001, 0.00001, 0.00001, 0.00002, 0.00002, 0.00002,
        0, 0, 0, 0, 0, 0, 0.00001, 0.00001, 0.00002, 0.00002, 0.00002, 0.00003,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
    ],
    'dcPower': [
        0, 0, 0, 0, 0, 0, 9e-8, 1e-8, 1e-8, 0.00000841, 0.00002209, 0.000025,
        0, 0, 0, 0, 0, 0, 1e-8, 1e-8, 1e-8, 4e-8, 4e-8, 4e-8, 0.00006274, 0.00,
        0, 0, 0, 0, 0, 0, 1e-8, 1e-8, 4e-8, 4e-8, 4e-8, 9e-8, 0.00041209, 0.00,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
    ]
}
```

```
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0  
]  
{
```



Quick Observation

Looking at Setup B (the second half of the dataset), your recorded `dcVoltage` , `dcCurrent` , and `dcPower` are entirely zeros. If this was intentional (e.g., a control group or a system that failed to generate power), the dataset is good to go! If Setup B was supposed to have numbers, you might want to double-check your data collection script or source spreadsheet.